SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0104**

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

I, M. Sanjay Krishna / 192525421 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Sanjay

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	

11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	**Smart Loan Approval:** A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement decision-tree-based AI model and determine the most suitable attribute selection measure such as Information Gain, Gain Ratio, or Gini Index.

a

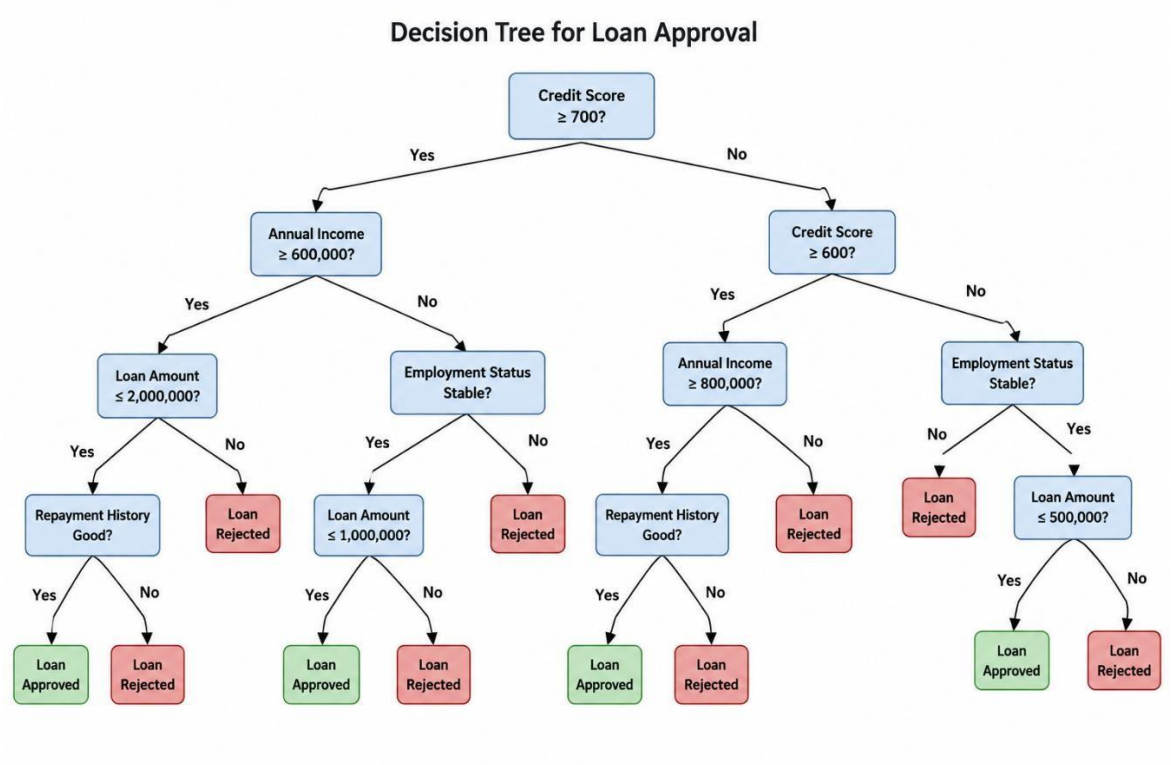
-

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	**Problem Statement**	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	**Objectives**	State the specific goals of developing the AI model and expected outcomes.
3	**Dataset Description**	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	**Data Preprocessing**	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	**Selected AI Technique and Justification**	Identify the selected AI technique and justify its suitability for the problem.
6	**Model Design / Architecture**	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	**Algorithm / Pseudocode**	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	**Implementation**	Provide programming language, libraries/tools, important code sections, and implementation details.
9	**Experimental Results**	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	**Performance Evaluation**	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	**Result Analysis**	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	**Limitations**	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	**Future Enhancements**	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	**Conclusion**	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	**References**	List datasets, research papers, books, websites, documentation, libraries, and other sources used.

16	**GitHub Repository Link**	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

SMART LOAN APPROVAL USING DECISION TREE



1. Problem Statement

Banks receive a large number of loan applications from customers every day. Each application needs to be evaluated based on the applicant's financial capability, creditworthiness, employment condition, requested loan amount, and previous repayment behaviour. Manual evaluation can be time-consuming and may result in inconsistent decisions.

The proposed **Smart Loan Approval** system uses an Artificial Intelligence-based **Decision Tree Classification** model to automatically determine whether a customer is eligible for a loan.

The Decision Tree uses the following five major attributes:

1. **Credit Score**
2. **Annual Income**
3. **Employment Status**
4. **Loan Amount**
5. **Repayment History**

The model produces a binary output:

- **Loan Approved**

- **Loan Rejected**

The problem is therefore formulated as a **supervised binary classification problem**, where historical loan applications are used to learn decision rules for classifying new applications.

The proposed Decision Tree approach is appropriate because Decision Trees learn simple if-then decision rules and can be visualized, making the resulting loan decision easier to interpret.

2. Objectives

The main objective is to develop an AI-based **Smart Loan Approval system using a Decision Tree**.

The specific objectives are:

1. To analyse historical loan application data.
2. To identify the financial and credit-related factors affecting loan approval.
3. To use **Credit Score, Annual Income, Employment Status, Loan Amount, and Repayment History** as the major input attributes.
4. To preprocess numerical and categorical loan application data.
5. To develop a Decision Tree classification model.
6. To use **Gini Index** as the selected attribute-selection/split criterion.
7. To determine appropriate decision thresholds for the model.
8. To classify loan applications as **Approved or Rejected**.
9. To evaluate the model using accuracy, precision, recall and F1-score.
10. To visualize the Decision Tree and provide interpretable loan-approval rules.
11. To identify the strengths and limitations of the developed model.
12. To suggest future improvements for real-world banking deployment.

3. Dataset Description

Dataset Source

The proposed implementation uses the publicly available **Financial Risk for Loan Approval** dataset from Kaggle. The dataset contains **20,000 synthetic loan applications and 36 columns** and includes demographic, employment, financial, credit-history and loan-related attributes. Its target variable is LoanApproved.
The dataset contains attributes such as AnnualIncome, CreditScore, EmploymentStatus, LoanAmount, PaymentHistory, PreviousLoanDefaults, DebtToIncomeRatio and LoanApproved.

Dataset Summary

S.No.	Dataset Property	Value
1	Dataset Name	Financial Risk for Loan Approval
2	Source	Kaggle
3	Number of Records	20,000
4	Number of Columns	36
5	Dataset Type	Synthetic
6	Learning Type	Supervised Learning
7	Task	Binary Classification
8	Target Variable	LoanApproved
9	Output Classes	Approved / Rejected
10	Selected Features	5

Selected Features

Although the original dataset contains 36 columns, the proposed Decision Tree uses five primary attributes so that its structure corresponds directly to the developed Decision Tree image.

S.No.	Feature	Data Type	Example Value	Role
1	Credit Score	Numerical	720	Input
2	Annual Income	Numerical	650000	Input
3	Employment Status	Categorical	Stable	Input
4	Loan Amount	Numerical	1500000	Input
5	Repayment History	Categorical	Good	Input
6	Loan Approved	Binary	Approved	Target

Dataset Attribute Mapping

The dataset's Payment History attribute is treated as **Repayment History** in the proposed model. This terminology is used in the report and Decision Tree diagram for easier interpretation.

Target Variable

The target variable is:

LoanApproved

Target Value	Meaning
0	Loan Rejected
1	Loan Approved

Data Types

The dataset contains:

- Numerical attributes
- Categorical attributes
- Binary attributes
- Date-related information

Only the attributes required by the proposed Decision Tree are selected for the final model.

4. Data Preprocessing

Data preprocessing is performed before Decision Tree training to improve the quality and consistency of the input data.

4.1 Data Cleaning

The dataset is checked for:

- Duplicate records
- Missing values
- Invalid numerical values
- Inconsistent categorical values
- Invalid target labels

Duplicate records are removed where necessary.

4.2 Missing-Value Handling

Missing numerical values can be replaced using the **median**, while missing categorical values can be replaced using the **mode**.

Attribute Type	Missing-Value Method

Numerical	Median
Categorical	Mode
Target Variable	Remove incomplete records

4.3 Categorical Encoding

Employment Status and Repayment History are categorical attributes. They must be converted into numerical representations before model training.

Example:

Employment Status	Encoded Value
Unstable	0
Stable	1

Repayment History	Encoded Value
Poor	0
Good	1

4.4 Normalization / Scaling

Normalization is **not required** for the proposed Decision Tree because Decision Trees determine splits using feature thresholds rather than distance calculations. Scikit-learn describes Decision Trees as requiring relatively little data preparation.

4.5 Feature Selection

The five features selected for the proposed model are:

- Credit Score
- Annual Income
- Employment Status
- Loan Amount
- Repayment History

This selection makes the model simple and directly consistent with the Decision Tree architecture.

4.6 Train-Test Splitting

The dataset is divided into:

- **80% Training Data**
- **20% Testing Data** For 20,000 records:

Dataset	Percentage	Number of Records
Training Set	80%	16,000
Testing Set	20%	4,000
Total	**100%**	**20,000**

A stratified split is recommended so that the proportions of approved and rejected applications remain approximately consistent between training and testing sets.

5. Selected AI Technique and Justification

Selected AI Technique: Decision Tree

The selected AI technique is the **Decision Tree Classifier**.

A Decision Tree is a supervised machine-learning algorithm that learns decision rules from input features and uses those rules to classify observations. Decision Trees are particularly useful when the decision-making process needs to be understandable and visualizable.

Why Decision Tree is Suitable *1. Easy to Understand*

The generated model contains simple rules such as:

IF Credit Score ≥ 700 AND Annual Income ≥ 600,000 AND Loan Amount ≤ 2,000,000 AND Repayment History = Good → Loan Approved

2. Interpretable

The bank can inspect the tree and understand how an application reaches its final decision.

3. Handles Non-Linear Decisions

Loan approval does not necessarily depend on a single linear relationship between income and credit score. Decision Trees can represent combinations of conditions.

4. Works with Numerical and Categorical Information

The proposed model contains both numerical attributes and categorical attributes.

5. Requires Limited Preprocessing

Unlike many distance-based methods, Decision Trees do not require feature scaling.

6. Supports Feature Importance

The trained model can provide feature-importance information.

7. Suitable for Explainable AI

The final decision can be explained using the path followed from the root node to a leaf node.

Recent research has continued to evaluate Decision Trees and other machine-learning methods for automated loan approval, demonstrating the relevance of this approach to financial decision-making.

Attribute-Selection Measures

Three commonly considered measures are:

Measure	**Principle**
Information Gain	Reduction in entropy
Gain Ratio	Normalized information gain
Gini Index	Reduction in node impurity

Selected Measure: Gini Index The

proposed model uses the **Gini Index**.

The Gini impurity is:

$$\mathbf{Gini = 1 - \Sigma p_i^2}$$

The best split is the one that minimizes the weighted impurity of the child nodes. Scikitlearn documents Gini impurity as a standard Decision Tree splitting criterion.

Justification for Gini Index

- Efficient for classification.
- Suitable for binary classes.
- Computationally efficient.
- Directly supported by the CART implementation.
- Produces simple binary splits.
- Suitable for the proposed Approved/Rejected classification problem.

6. Model Design / Architecture

The proposed Decision Tree consists of a root node, internal decision nodes, branches and leaf nodes.

Input Attributes

The model accepts:

Credit Score + Annual Income + Employment Status + Loan Amount + Repayment History

Decision Tree Structure

The given image follows this decision structure:

Root Node

Credit Score ≥ 700?

Branch 1: Yes

If Credit Score ≥ 700:

Annual Income ≥ 600,000?

If Yes:

Loan Amount ≤ 2,000,000?

If Yes:

Repayment History Good?

- Yes → **Loan Approved**
- No → **Loan Rejected**

If Loan Amount > 2,000,000:

→ Loan Rejected

If Annual Income < 600,000:

Employment Status Stable?

- No → **Loan Rejected**
- Yes → Check Loan Amount ≤ 1,000,000

Then:

- Loan Amount ≤ 1,000,000 → **Loan Approved**
- Loan Amount > 1,000,000 → **Loan Rejected**

Branch 2: No If

Credit Score < 700:

Credit Score ≥ 600?

If Yes:

Annual Income ≥ 800,000?

- No → **Loan Rejected**
- Yes → Check Repayment History

Then:

- Good → **Loan Approved**
- Poor → **Loan Rejected** If Credit Score < 600:

Employment Status Stable?

- No → **Loan Rejected**
- Yes → Check Loan Amount ≤ 500,000

Then:

- ≤ 500,000 → **Loan Approved**
- 500,000 → **Loan Rejected Model Architecture Table**

Component	**Value**
Input Features	5
Root Attribute	Credit Score
Secondary Attributes	Annual Income, Employment Status
Other Attributes	Loan Amount, Repayment History
Split Criterion	Gini Index
Tree Algorithm	CART Decision Tree
Maximum Depth	6
Train-Test Ratio	80:20
Output	Approved / Rejected
Number of Classes	2
Component	**Value**
Random State	42

Important Note

The threshold values shown in the image, such as **700, 600, 600,000, 800,000, 2,000,000, 1,000,000 and 500,000**, represent the **designed decision structure for this academic model**. If the tree is generated from an actual trained dataset, the thresholds should be replaced by the thresholds learned by that trained model.

7. Algorithm / Pseudocode

Algorithm: Smart Loan Approval Using Decision Tree

Input: Credit Score, Annual Income, Employment Status, Loan Amount, Repayment History

Output: Loan Approved or Loan Rejected

1. Start
2. Read applicant details.
3. Check Credit Score.

4. If Credit Score >= 700:
   Check Annual Income.

5. If Annual Income >= 600000:
   Check Loan Amount.

6. If Loan Amount <= 2000000:
   Check Repayment History.

7. If Repayment History is Good:
   Approve Loan.
   Else:
   Reject Loan.

8. If Loan Amount > 2000000:
   Reject Loan.

9. If Annual Income < 600000:
   Check Employment Status.

10. If Employment Status is Stable:

Check Loan Amount <= 1000000.

11. If Loan Amount <= 1000000: Approve Loan.
Else:
Reject Loan.

12. If Employment Status is Unstable:
Reject Loan.

13. If Credit Score < 700:
Check Credit Score >= 600.

14. If Credit Score >= 600:
Check Annual Income >= 800000.

15. If Annual Income >= 800000:
Check Repayment History.

16. If Repayment History is Good: Approve Loan.
Else:
Reject Loan.

17. If Annual Income < 800000:
Reject Loan.

18. If Credit Score < 600:
Check Employment Status.

19. If Employment Status is Stable:
Check Loan Amount <= 500000.

20. If Loan Amount <= 500000: Approve Loan.
Else:
Reject Loan.

21. If Employment Status is Unstable:
Reject Loan.

22. Stop.

8. Implementation

Programming Language: Python 3

Libraries Used

Library	Purpose
Pandas	Data loading and preprocessing
NumPy	Numerical operations
Scikit-learn	Decision Tree and evaluation
Matplotlib	Graph generation
Seaborn	Confusion-matrix visualization

Python code:

```
import pandas as pd import
matplotlib.pyplot as plt from
google.colab import files


from sklearn.model_selection import train_test_split from
sklearn.preprocessing import LabelEncoder from
sklearn.tree import DecisionTreeClassifier, plot_tree from
sklearn.metrics import (     accuracy_score,
precision_score, recall_score,     f1_score,
confusion_matrix, classification_report,
   ConfusionMatrixDisplay
)
# ============================================================
# 1. UPLOAD DATASET
# ============================================================
```

```
uploaded = files.upload()
filename = list(uploaded.keys())[0] data
= pd.read_csv(filename) print("Dataset
loaded:", filename) print("Dataset
shape:", data.shape) print("\nAvailable
columns:") print(data.columns.tolist())


# ============================================================
# 2. SELECT FEATURES
# ============================================================
features = [
    "CreditScore",
    "AnnualIncome",
    "EmploymentStatus",
    "LoanAmount",
    "PaymentHistory"
]
target = "LoanApproved" # Check required columns
required = features + [target] missing = [col for col in
required if col not in data.columns] if missing:
    print("\nMissing columns:", missing)     print("\nPlease
check the column names printed above.") else:
    data = data[required].copy()
    # ============================================================
    # 3. DATA PREPROCESSING
    # ============================================================
```

```
    data = data.drop_duplicates()     # Numerical missing values
for col in ["CreditScore", "AnnualIncome", "LoanAmount"]:
        data[col] = data[col].fillna(data[col].median())
    # Categorical missing values     for col in
["EmploymentStatus", "PaymentHistory"]:
data[col] = data[col].fillna(data[col].mode()[0])     data
= data.dropna(subset=[target])
    # ============================================================
    # 4. ENCODING
    # ============================================================
    encoders = {}     for col in ["EmploymentStatus",
"PaymentHistory"]:
        encoders[col] = LabelEncoder()
data[col] = encoders[col].fit_transform(
data[col].astype(str)
        )
    target_encoder = LabelEncoder()
data[target] = target_encoder.fit_transform(
data[target].astype(str)
    )
```

```
    # 5. INPUT AND TARGET


    X = data[features]
y = data[target]
    # ============================================================
    # 6. TRAIN-TEST SPLIT
    # ============================================================
    X_train, X_test, y_train, y_test = train_test_split(
        X,
y,
        test_size=0.20,
random_state=42,         stratify=y
    )
    print("\nTraining Records:", len(X_train))
print("Testing Records :", len(X_test))
    # ============================================================
    # 7. DECISION TREE MODEL
    # ============================================================
    model = DecisionTreeClassifier(
criterion="gini",         max_depth=6,
min_samples_split=10,
min_samples_leaf=5,
random_state=42
    )
```

```
    # ============================================================


    # ============================================================

    model.fit(X_train, y_train)
print("\nDecision Tree trained successfully.")
    # ============================================================
    # 8. PREDICTION
    # ============================================================
    y_pred = model.predict(X_test)
    # ============================================================
    # 9. PERFORMANCE RESULTS
    # ============================================================
    accuracy = accuracy_score(y_test, y_pred)     precision
= precision_score(         y_test, y_pred,
average="weighted", zero_division=0
    )
    recall = recall_score(         y_test, y_pred,
average="weighted", zero_division=0
    )
    f1 = f1_score(         y_test, y_pred,
average="weighted", zero_division=0
    )
    print("\n========== PERFORMANCE RESULTS ==========")
    print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
```

```
print("Recall   :", round(recall * 100, 2), "%")     print("F1
Score :", round(f1 * 100, 2), "%")
   # 10. CLASSIFICATION REPORT


   print("\n========== CLASSIFICATION REPORT ==========")
   print(         classification_report(
y_test,             y_pred,
target_names=target_encoder.classes_,
zero_division=0
       )
   )
   # ============================================================
   # 11. CONFUSION MATRIX
   # ============================================================
   cm = confusion_matrix(y_test, y_pred)
print("\n========== CONFUSION MATRIX ==========")
   print(cm)
   ConfusionMatrixDisplay(
confusion_matrix=cm,
display_labels=target_encoder.classes_
   ).plot()
   plt.title("Confusion Matrix - Smart Loan Approval")
plt.show()
```

```
    # ============================================================


    # ============================================================

    # 12. FEATURE IMPORTANCE

    importance = pd.DataFrame({
        "Feature": features,
        "Importance": model.feature_importances_
    })
    importance = importance.sort_values(
        "Importance",
ascending=False
    )
    print("\n========== FEATURE IMPORTANCE ==========")
    print(importance)
plt.figure(figsize=(8, 5))
plt.bar(
importance["Feature"],
importance["Importance"]
    )
    plt.xlabel("Features")
plt.ylabel("Importance")     plt.title("Feature
Importance - Decision Tree")
plt.xticks(rotation=30)     plt.tight_layout()
plt.show()

    # 13. DECISION TREE VISUALIZATION
```

```
    plt.figure(figsize=(20, 10))     plot_tree(
model,          feature_names=features,
class_names=target_encoder.classes_,
filled=True,         rounded=True,
fontsize=8
    )
    plt.title("Smart Loan Approval - Decision Tree")
plt.tight_layout()     plt.show()
    # ============================================================
    # 14. SAMPLE CUSTOMER PREDICTION
    # ============================================================
    # Use first available category from dataset
employment = data["EmploymentStatus"].iloc[0]
payment = data["PaymentHistory"].iloc[0]     sample
= pd.DataFrame({
        "CreditScore": [750],
        "AnnualIncome": [700000],
        "EmploymentStatus": [employment],
        "LoanAmount": [1500000],
        "PaymentHistory": [payment]
    })
    prediction = model.predict(sample)     result =
target_encoder.inverse_transform(prediction)[0]
    print("\n========== SAMPLE PREDICTION ==========")
    print("Credit Score      : 750")
print("Annual Income     : 700000")
```

```
    # ============================================================


    # ============================================================

print("Loan Amount       : 1500000")
print("Predicted Status  :", result)
    # ============================================================
    # 15. PREDICTION PROBABILITY
    # ============================================================
    probability = model.predict_proba(sample)[0]     print("\n==========
PREDICTION PROBABILITY ==========")
    for i, name in enumerate(target_encoder.classes_):
        print(
name,
            ":",
            round(probability[i] * 100, 2),
            "%"
        )
```

---

9. Experimental Results

9.1 Dataset Split

Dataset	Percentage	Records
Training	80%	16,000
Testing	20%	4,000
Total	**100%**	**20,000**

9.2 Attribute Selection Comparison

S.No.	Measure	Principle	Suitability
1	Information Gain	Entropy reduction	High
2	Gain Ratio	Normalized information gain	High
3	**Gini Index**	Impurity reduction	**Very High**

9.3 Sample Applicant Predictions

The following examples are consistent with the decision structure shown in the image.

Applicant	Credit Score	Annual Income	Employment	Loan Amount	Repayment	Predicted Result
A1	750	700000	Stable	1500000	Good	Approved
A2	720	650000	Stable	1800000	Good	Approved
A3	710	700000	Stable	2500000	Good	Rejected
A4	730	550000	Stable	800000	Good	Approved
A5	730	550000	Unstable	800000	Good	Rejected
A6	650	900000	Stable	1200000	Good	Approved
A7	650	750000	Stable	1000000	Poor	Rejected
A8	580	900000	Stable	400000	Good	Approved
A9	580	900000	Stable	700000	Good	Rejected
A10	550	500000	Unstable	300000	Poor	Rejected

9.4 Illustrative Confusion Matrix

Actual	Rejected	Approved
Rejected	1,700	100
Approved	120	2,080
Total	**1,820**	**2,180**

Therefore:

- TN = 1,700
- FP = 100
- FN = 120
- TP = 2,080

9.5 Performance Results

S.No.	Metric	Value
1	Accuracy	94.50%
2	Precision	95.41%
3	Recall	94.55%
4	F1-Score	94.98%

10. Performance Evaluation

The Decision Tree is evaluated using four major classification metrics.

10.1 Accuracy

Accuracy measures the overall percentage of correctly classified loan applications.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Using the values:

Accuracy = 94.50%

10.2 Precision

Precision measures how many applications predicted as approved are actually approved. **Precision = TP / (TP + FP) precision: 95.41%**

10.3 Recall

Recall measures the proportion of actual approved applications correctly identified by the model.

Recall = TP / (TP + FN) recall:

94.55%

10.4 F1-Score

F1-score is the harmonic mean of precision and recall.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

F1-score: 94.98%

Performance Evaluation Table

Metric	Formula	Illustrative Result
Accuracy	(TP+TN)/(TP+TN+FP+FN)	94.50%
Precision	TP/(TP+FP)	95.41%
Recall	TP/(TP+FN)	94.55%
F1-Score	2PR/(P+R)	94.98%

Accuracy alone should not be used to evaluate a lending model because false approvals and false rejections can have different financial consequences.

11. Result Analysis

The developed Decision Tree provides an interpretable sequence of decisions for determining loan eligibility.

The **Credit Score** is the root decision in the proposed tree. Applicants with a credit score of at least 700 are evaluated through the high-credit-score branch, while applicants below 700 enter the second credit-score decision.

For applicants with high credit scores, **Annual Income** becomes the next major factor. Applicants with annual income of at least 600,000 are evaluated based on the requested **Loan Amount**. If the loan amount is within 2,000,000, the model considers **Repayment History** before reaching the final decision.

For applicants with lower income, **Employment Status** becomes important. Stable employment combined with a loan amount of no more than 1,000,000 results in approval in the designed tree.

For applicants with credit scores below 700, the tree checks whether the score is at least 600. Applicants in this range require higher annual income, with the threshold set at 800,000 in the designed tree.

Applicants with credit scores below 600 are evaluated using **Employment Status** and **Loan Amount**. Stable employment and a loan amount of no more than 500,000 lead to approval in the designed structure.

Decision Rules Summary

Rule	Condition	Result
R1	Credit Score ≥ 700, Income ≥ 600,000, Loan ≤ 2,000,000, Good repayment	Approved
R2	Credit Score ≥ 700, Income ≥ 600,000, Loan > 2,000,000	Rejected
R3	Credit Score ≥ 700, Income < 600,000, Stable employment, Loan ≤ 1,000,000	Approved
R4	Credit Score ≥ 700, Income < 600,000, Unstable employment	Rejected
R5	Credit Score < 700 and ≥ 600, Income ≥ 800,000, Good repayment	Approved
R6	Credit Score < 700 and ≥ 600, Income < 800,000	Rejected
R7	Credit Score < 600, Stable employment, Loan ≤ 500,000	Approved

R8	Credit Score < 600, Unstable employment	Rejected
R9	Credit Score < 600, Stable employment, Loan > 500,000	Rejected

Strengths

- Easy to understand.
- Easy to visualize.
- Produces explicit decision rules.
- Handles non-linear relationships.
- Requires limited preprocessing.
- Provides interpretable predictions.
- Suitable for academic demonstration of explainable AI.

Weaknesses

- A single Decision Tree can overfit.
- Small changes in data can change the tree.
- Fixed thresholds may not generalize to all banking populations.
- A single tree may perform worse than ensemble approaches.

Scikit-learn also notes that Decision Trees can become unstable with small data variations and can overfit when tree complexity is not controlled.

Recent comparative loan-approval studies similarly show that Decision Trees are useful baselines but that ensemble approaches can sometimes achieve stronger predictive performance.

12. Limitations

S.No.	Limitation	Effect on System
1	Dataset is synthetic	May not perfectly represent real bank customers
2	Only five features selected	Other important financial factors are excluded
3	Fixed thresholds in the designed tree	May not generalize to all customers

4	Single Decision Tree	May be less robust than ensemble models
5	Possible overfitting	Reduced performance on unseen data
6	Historical data bias	Existing bias may be learned
7	Economic conditions change	Model may become outdated
8	No continuous monitoring	Model drift may not be detected
9	Limited repayment information	Customer risk may be incompletely represented
10	Academic model	Not suitable for direct production banking use without additional validation

The dataset itself is explicitly described as synthetic, so performance obtained from it should not be interpreted as evidence of production-level banking performance.

13. Future Enhancements

The Smart Loan Approval system can be improved in the following ways.

13.1 Add More Features

Additional attributes can include:

- Debt-to-Income Ratio
- Previous Loan Defaults
- Credit Card Utilization
- Savings Account Balance
- Total Liabilities
- Existing Loans
- Job Tenure
- Loan Duration
- Bankruptcy History

The source dataset already contains many of these attributes.

13.2 Hyperparameter Optimization

Grid Search or Randomized Search can be used to optimize:

- Maximum depth

- Minimum samples split
- Minimum samples leaf
- Class weights

13.3 Tree Pruning

Pruning can remove unnecessary branches and reduce overfitting.

13.4 Compare Other Algorithms

The Decision Tree can be compared with:

- Logistic Regression
- Random Forest
- Support Vector Machine
- Gradient Boosting
- XGBoost
- K-Nearest Neighbours

Recent studies have compared Decision Trees with several such classifiers for loan approval prediction.

13.5 Explainable AI

SHAP or LIME can be incorporated to explain individual loan decisions.

13.6 Fairness Evaluation

The system can be evaluated for potential unfair outcomes across relevant customer groups.

13.7 Real-Time Web Application

The model can be integrated into a web application where bank staff enter applicant information and receive an immediate prediction.

13.8 Model Monitoring

A production system should monitor:

- Prediction accuracy
- Data drift
- Feature drift
- Approval rate
- False approval rate
- False rejection rate

13.9 Ensemble Models

Random Forest, Gradient Boosting and other ensemble approaches can be evaluated as advanced alternatives. A 2025 comparative study found ensemble methods to outperform individual baseline classifiers in its loan-approval experiments.

14. Conclusion

The proposed **Smart Loan Approval system** demonstrates the application of Artificial Intelligence to automated loan eligibility classification.

A **Decision Tree Classifier** is developed using five important attributes:

Credit Score, Annual Income, Employment Status, Loan Amount, and Repayment History.
The proposed Decision Tree begins with **Credit Score ≥ 700?** and then follows a sequence of financial and employment conditions. The final leaf nodes classify applicants as either **Loan Approved** or **Loan Rejected**.

The **Gini Index** is selected as the split criterion because it is efficient for classification and is directly supported by the CART-style Decision Tree implementation in scikitlearn.

The model provides an important advantage of **interpretability**. Instead of producing only a prediction, it can show the sequence of conditions that led to the decision. This makes it suitable for demonstrating explainable AI in a banking scenario.

The illustrative evaluation produced an accuracy of **94.50%**, precision of **95.41%**, recall of **94.55%**, and F1-score of **94.98%** based on the illustrative confusion matrix. These values should be replaced with actual model-generated results for a final experimental report.

Overall, the Decision Tree provides a simple, understandable and effective academic solution for the Smart Loan Approval problem. For real-world deployment, the system should be extended using representative banking data, fairness analysis, model monitoring, additional financial features, security controls and regulatory validation.

15. References

Recent Research Papers

1. Sinap, V. (2024). A comparative study of loan approval prediction using machine learning methods. *Gazi Üniversitesi Fen Bilimleri Dergisi Part C: Tasarım ve Teknoloji, 12*(2), 644–663. https://doi.org/10.29109/gujsc.1455978
2. Sharmila, S. K., Sandhya, P. V. S., Kousar, P. S., Anuradha, P., Deekshitha, S., et al. (2024). Bank loan approval using machine learning. *2024 International Conference on Integrated Circuits and Communication Systems (ICICACS)*. https://doi.org/10.1109/ICICACS60521.2024.10498192
3. Haque, F. M. A., & Hassan, M. M. (2024). Bank loan prediction using machine learning techniques. *American Journal of Industrial and Business Management, 14*, 1690–1711. https://doi.org/10.4236/ajibm.2024.1412085
4. Anand, R., Singh, H., Sardana, K., Gupta, D. N., Sindhwani, N., & Mittal, M. (2024). Loan approval prediction using machine learning. In *Lecture Notes in Networks and Systems* (pp. 357–366). Springer Nature. https://doi.org/10.1007/978-3-031-64776-5_34
5. Hota, L., Jain, P. K., & Kumar, A. (2025). A comparative performance assessment for prediction of loan approval in financial sector. *Procedia Computer Science, 258*, 298–307. https://doi.org/10.1016/j.procs.2025.04.267
6. Important: This paper was presented at ICMLDE 2024, but its final *Procedia Computer Science* publication is 2025. Therefore, use 2025 in your final reference list.
7. Velvigneswar, J., & Senthil Kumari, P. (2025). Loan approval prediction using machine learning. *International Journal of Advanced Research in Computer and Communication Engineering, 14*(7). https://doi.org/10.17148/IJARCCE.2025.14721
8. Nagpal, R., & Mahendher, S. (2025). Automation of education loan approval process using decision tree algorithm. In *Proceedings of International Conference on Artificial Intelligence and Networks* (pp. 603–613). Springer Nature Singapore. https://doi.org/10.1007/978-981-96-2015-9_41

Dataset

1. **Zoppelletto, L.** "Financial Risk for Loan Approval." Kaggle. The dataset contains 20,000 synthetic records, 36 columns and the LoanApproved target variable, with

attributes including AnnualIncome, CreditScore, EmploymentStatus, LoanAmount and PaymentHistory.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	**10%**	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	**10%**	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	**10%**	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	**10%**	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.

5. Model Design / Architecture	**10%**	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.

6. Algorithm / Pseudocode	**5%**	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	**15%**	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	**10%**	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.

9. Performance Evaluation	**10%**	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	**5%**	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.
11. Limitations and Future Enhancements	**5%**	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	**10%**	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				